

ЛР №8: Экосистема TanStack: Роутинг и Серверное состояние

Цель работы: Модернизация приложения созданного в ЛР 7 путем замены ручного управления состоянием и навигацией на экосистему TanStack. Переход от императивного `useEffect` к декларативному серверному состоянию и внедрение типобезопасного роутинга.

1. Рефакторинг навигации (TanStack Router)

Вместо ручного переключения компонентов в `App.tsx`, необходимо реализовать файловый роутинг.

- **Дерево маршрутов:**
 - `__root.tsx`: Общий макет (Layout) с навигационной панелью (Navbar).
 - `index.tsx`: Главная страница (Landing).
 - `login.tsx`: Страница авторизации.
 - `catalog.tsx`: Страница каталога (защищенный маршрут).
 - `product.$id.tsx`: Страница деталей конкретного товара.
- **Защита маршрутов (Auth Guard):** реализовать логику защиты в методе `beforeLoad` для маршрута каталога. Если пользователь не авторизован — выполнять `throw redirect({ to: '/login' })`.
- **Безопасная навигация:** использовать компонент `<Link>` для всей навигации.

2. Управление данными (TanStack Query)

Полностью исключить использование `useEffect` для запросов и `useReducer` для хранения серверных данных.

- **Замена Read:** Использовать хук `useQuery` для получения списка товаров.
 - Настроить `staleTime: 60000` (1 минута) и `gcTime: 300000` (5 минут).
 - Реализовать обработку состояний `isLoading` и `isError` с выводом соответствующих UI-компонентов.
- **Замена CRUD (Create, Update, Delete):** Использовать хук `useMutation`.
 - **Инвалидация:** После успешного выполнения мутации (добавление, изменение или удаление) вызывать `queryClient.invalidateQueries({ queryKey: ['products'] })` для автоматического обновления списка.
 - **Оптимистичное обновление:** Реализовать оптимистичное удаление товара — товар должен исчезать из списка мгновенно, до завершения сетевого запроса, с откатом состояния при ошибке.

3. Работа с URL и Search Params

- **Валидация поиска:** Реализовать фильтрацию товаров через Search Params (например, `?category=smartphones`).
- Использовать метод `validateSearch` в конфигурации маршрута совместно с **Zod** для проверки параметров URL.
- Параметры поиска должны быть строго типизированы и доступны в компоненте через хук `useSearch`.

5. Валидация API-ответов (Zod)

- Внедрить валидацию данных прямо в функцию запроса (`queryFn`).
- Перед тем как данные попадут в кэш TanStack Query, они должны пройти через `ProductSchema.parse(data)`.
- Реализовать централизованную обработку ошибок валидации, чтобы при несоответствии типов пользователь видел уведомление «Ошибка структуры данных», а не белый экран.

Контрольные вопросы

1. Почему использование `useQuery` предпочтительнее связки `useEffect + useState` для получения данных с сервера?
2. В чем разница между `staleTime` и `gcTime`? Что произойдет, если пользователь вернется на страницу через 30 секунд при `staleTime: 60000`?
3. Опишите жизненный цикл мутации при добавлении товара. Зачем нужен вызов `invalidateQueries` в методе `onSuccess`?
4. Какую проблему решает TanStack Router в сравнении с обычными строковыми путями (например, в React Router v6)?
5. Зачем нужен файл `routeTree.gen.ts` и как он помогает избежать runtime-ошибок при навигации?
6. Как работает механизм `beforeLoad` в TanStack Router и почему он надежнее проверки авторизации внутри `useEffect` компонента?
7. Если сервер пришлет поле `price` в виде строки вместо числа, на каком этапе Zod прервет выполнение программы и как это защитит UI?
8. Что такое «Оптимистичное обновление» и какие методы хука `useMutation` необходимы для его реализации?
9. С помощью DevTools продемонстрировать переход данных из состояния `Fresh` в `Stale`.